
User:

is there any tricky or most asked questions on functional interface or not that much imp?

ChatGPT:

Functional interfaces **are important** in Java interviews—especially when you're being evaluated for Java 8+ knowledge. While they may not always be the centerpiece, **interviewers love to test your depth** with subtle or tricky questions around them. Here's a breakdown:

✅ Why Functional Interfaces Are Important

- **Key to Lambda expressions** and method references.
 - Backbone of Java 8 features like Stream, Optional, and CompletableFuture.
 - Used heavily in **clean code**, **event-driven programming**, and **Spring's functional style**.
-

🔥 Most Asked / Tricky Questions on Functional Interfaces

1. What is a functional interface? Can an interface with default methods be functional?

- **✅ Expected Answer:**
A functional interface has **only one abstract method**. Yes, it **can have default and static methods**, as long as only one abstract method exists.

2. Can a functional interface extend another interface?

- **✅ Expected Answer:**
Yes. But the **combined abstract methods from all super interfaces must not exceed one**.
Tricky case:

```
java
interface A { void m(); }
interface B extends A { void m(); } // still functional
```

3. Can we annotate an interface with @FunctionalInterface if it has multiple abstract methods?

- **✅ Expected Answer:**
No. It will cause a **compilation error**. The annotation is not required, but it's used for **compile-time checking**.

4. Is Comparator<T> a functional interface?

- **✅ Answer:** Yes. It has one abstract method: compare(T o1, T o2). The others are default.

5. Difference between Function<T, R> and Predicate<T>?

- **✅ Function<T, R>** takes T and returns R.
- **✅ Predicate<T>** takes T and returns boolean.

6. Can you create your own functional interface?

- ☒ Yes, by defining an interface with one abstract method. Optional: Use `@FunctionalInterface`.

7. What happens if you add a second abstract method in a functional interface?

- ☒ If annotated with `@FunctionalInterface`, **compiler error**.
- Without annotation, it compiles but **won't work with lambdas**.

8. Can a functional interface extend a non-functional interface?

- ☒ Yes, **if** after extending, the total number of abstract methods is still one.

Pro Tips

- **Use `@FunctionalInterface`** always — for safety and clarity.
- Think in terms of **SAM** (Single Abstract Method).
- Lambdas can only be assigned to **functional interfaces**.
- Avoid making your interface "accidentally" non-functional by adding another method.

Want a few quick Java 8 lambda + functional interface combo questions next? Or how Spring uses functional interfaces (like in routes or bean configuration)?

User:

package Beginner.java8; @java.lang.FunctionalInterface public interface FunctionalInterface { int apply(int a, int b); //int apply1(int a, int b);//  Compilation Error! Multiple non-overriding abstract methods found in interface default void defaultMethod() { } static void staticMethod() //Static methods in interfaces should have a body { } } interface B extends FunctionalInterface{ int apply(int a, int b); int apply1(int a, int b); } but 2. Can a functional interface extend another interface? ☒ Expected Answer: Yes. But the combined abstract methods from all super interfaces must not exceed one. this is not giving error

ChatGPT:

You're catching onto a **subtle but very good point** — this looks contradictory on the surface, but it's actually a **Java nuance**.

Let's break it down step by step 

Your Code Breakdown

```
java
@FunctionalInterface
public interface FunctionalInterface {
    int apply(int a, int b);
}
```

- ☒ This is valid — a functional interface with one abstract method.